Session 08 Exercise: APIs, HTTP Requests + Environment Variables

Agentic AI Builders Fellowship | Phase 1 — Python for AI

Skill: Build an API client class and save results to JSON

Before You Start

> **Check 1** You need your Session 07 project folder open. That folder already has your virtual environment and your module structure. Today's exercise adds two new files to it.

Run these commands to confirm your environment is ready:

```
$ cd your_project_folder
$ source venv/bin/activate       # Mac/Linux
$ venv\Scripts\activate          # Windows
$ pip install requests python-dotenv
```

Expected output: Successfully installed requests-2.x.x python-dotenv-1.x.x (or "already satisfied" if already installed)

```
$ pip freeze > requirements.txt
```

This updates requirements.txt to include the two new libraries.

Key Concepts in Plain English

API (Application Programming Interface)

A set of rules that lets two programs talk to each other over the internet. Your Python code sends a request to a server. The server sends back data. You never need to know how the server works — you only need to know what URL to call and what it returns.

HTTP GET request

An instruction that says "send me data." Nothing on the server changes. In Python: requests.get(url) sends the request and gives you back a response object containing the server's reply.

JSON response

The data the server sends back, formatted as text that looks like a Python dictionary. response.json() converts it into a real Python dict you can read and work with — exactly like the JSON files you used in Session 06.

Status code

A number the server attaches to every response to tell you what happened. 200 means success. 404 means not found. 500 means the server broke. response.raise_for_status() checks this automatically and raises an exception if the code signals an error.

Environment variable

A named value stored outside your Python code — in a separate file called .env. You read it with os.environ.get("KEY_NAME"). The key never appears in your Python file. This protects it from being accidentally shared on GitHub.

API client class

A class whose job is to wrap all the communication with one external API. The __init__ method stores shared configuration (base URL, timeout). Each method does one job: geocode a city, fetch weather data, and so on. You learned to build classes in Session 05 — today you apply that knowledge to a real API.

What You Saw in the Demo

During the live session, the instructor built a three-file Weather Dashboard:

- weather_client.py: a WeatherClient class with __init__, geocode(), get_current_weather(), and fetch() methods
- result_saver.py: a ResultSaver class that appends weather results to a JSON file
- main.py: the orchestrator — creates one instance of each class, gets user input, calls methods, displays and saves results
- The API used was Open-Meteo (free, no key required) — the same pattern applies to any API
- .env.example showed the pattern for storing API keys outside your code

---

What You Will Build

You will add an API client class to your existing Phase 1 project. You will choose a free public API relevant to your project domain, build a class that wraps it (with __init__ and at least two instance methods), and save the results to a JSON file using a ResultSaver class.

When you are done, your project will have a real internet connection. It will fetch live data, display it in the terminal, and build a JSON history file that grows with every run.

Step-by-Step Build Guide

Step 1 — Choose your API

Pick one free public API from the list below. Read its documentation. Before writing any code, understand: What is the URL I call? What parameters does it need? What does the JSON response look like?

API	What it returns	URL
Open-Meteo	Real-time weather for any city	https://open-meteo.com/
RestCountries	Population, capital, currency, flag for any country	https://restcountries.com/

ZenQuotes	Motivational quotes	https://zenquotes.io/
JokeAPI	Clean programming or general jokes	https://jokeapi.dev/
Open Library	Book info by title or ISBN	https://openlibrary.org/developers/api
NewsAPI (free tier)	Top news headlines by category	https://newsapi.org/

Tip: paste the API URL into your browser first. If you see JSON text appear, the API is working.

Step 2 — Create your API client file

Create a new file in your project folder. Name it after your API, for example:

```
quotes_client.py   or   country_client.py   or   news_client.py
```

This file will contain one class. Use the WeatherClient structure from the demo as your model.

Step 3 — Write the class with __init__ and two methods

Your class must have:

1. An __init__ method that stores the base URL and timeout on self
2. A fetch method that takes one input (search term, country code, topic, etc.), validates it, calls the API, and returns the parsed JSON
3. A display method (or format_result method) that takes the API response and prints it in a readable format

Here is the skeleton to start from:

```
class YourAPIClient:
    BASE_URL = "https://the-api-url.com/endpoint"

    def __init__(self, timeout=10):
        self.timeout = timeout

    def fetch(self, search_term):
        # 1. Validate input
        # 2. Build params dict
        # 3. requests.get(self.BASE_URL, params=params,
timeout=self.timeout)
        # 4. response.raise_for_status()
        # 5. return response.json()

    def display(self, result):
        # Print the fields you want from the result dict
```

The leading self. on BASE_URL means it is a class-level constant. You access it in methods as self.BASE_URL.

Step 4 — Add input validation

Before your requests.get() call, check whether the input is usable. If it is empty or whitespace, print a message and return None. Do not call the API with bad input.

```
if not search_term or not search_term.strip():
```

```
    print("[YourClient] Input cannot be empty.")
    return None
```

Step 5 — Add error handling

Wrap your requests.get() call in try/except. Catch requests.exceptions.RequestException. If a network error occurs, print a message and return an empty dict or None.

```
try:
    response = requests.get(self.BASE_URL, params=params,
timeout=self.timeout)
    response.raise_for_status()
    return response.json()
except requests.exceptions.RequestException as error:
    print(f"[YourClient] Network error: {error}")
    return None
```

Step 6 — Create a ResultSaver class

Copy the ResultSaver class from the demo (result_saver.py) into your project, or write your own version. It must:

4. Store the output filepath in __init__
5. Have a save() method that appends one result record to the JSON file
6. Have a _load_existing() helper that reads the file (returns [] if it does not exist)

The save() method should build a dict with at least: a timestamp, your main data fields, and the search term that was used.

Step 7 — Wire it together in main.py

In your main.py (or equivalent entry point):

7. Import your client class and ResultSaver
8. Call load_dotenv() at the top (even if you have no keys yet — practice the pattern)
9. Create one instance of each class
10. Get user input with input()
11. Call client.fetch(user_input)
12. Call client.display(result)
13. Call saver.save(result)

Step 8 — Create .env and .env.example

.env.example is a template file that shows what keys are needed — with placeholder values. It is safe to commit to GitHub.

```
# .env.example
MY_API_KEY=your_key_here
```

.env is your actual file with real values. It must never be committed. Add it to .gitignore:

```
$ echo ".env" >> .gitignore
```

Even if your chosen API does not require a key, create both files as empty templates. You will need this habit in every session from here on.

Expected Output

When you run your main.py, the terminal should show something like this (exact content depends on your API):

```
$ python main.py

Enter your search term: stoicism

--------------------------------------------------
  RESULT
  "Do not indulge in dreams of what you do not have."
  — Marcus Aurelius
--------------------------------------------------

[ResultSaver] Saved to my_results.json (1 record(s) total)
Result saved at 2026-04-15T14:22:31
```

After two or three runs, open your JSON file. It should contain a list of records, one per run, each with a timestamp.

```
[
  {
    "timestamp": "2026-04-15T14:22:31",
    "search_term": "stoicism",
    "quote": "Do not indulge in dreams ...",
    "author": "Marcus Aurelius"
  },
  {
    "timestamp": "2026-04-15T14:25:10",
    "search_term": "motivation",
    ...
  }
]
```

Submission Rubric

Criterion	What to check	Result
API client class	A class with __init__, at least two instance methods, BASE_URL stored on class or self, timeout on self	Complete / Partial
Input validation	fetch() checks for empty/whitespace input before calling the API	Complete / Partial

Error handling	requests.get() wrapped in try/except RequestException; raise_for_status() present	Complete / Partial
ResultSaver + JSON output	Results append to a JSON file across multiple runs; file contains timestamp and all required fields	Complete / Partial
.env pattern	.env.example present and committed; .env in .gitignore; load_dotenv() called in main	Complete / Partial

Getting Unstuck

Problem: I get a KeyError when I try to read from response.json()

Fix: the JSON structure is different from what you expected. Run print(response.json()) first, before any parsing. Look at the actual keys the API returns. Then update your code to match the real structure.

Problem: ModuleNotFoundError: No module named 'requests'

Fix: your virtual environment is not activated, or you installed requests outside it. Run: source venv/bin/activate (Mac/Linux) or venv\Scripts\activate (Windows). Then: pip install requests python-dotenv.

Problem: My response.json() returns nothing useful / empty results

Fix: print the full URL you are calling by printing requests.Request("GET", self.BASE_URL, params=params).prepare().url — this shows the exact URL before you send it. Paste that URL into your browser. If the browser shows data, your Python code has the right URL. If not, check your params.

Problem: The .env file is not being read (os.environ.get returns None)

Fix: check three things. First — is .env in the same folder as main.py? Second — is load_dotenv() called before os.environ.get()? Third — is the key name spelled the same way in both .env and your Python file (case-sensitive)?

Problem: My class __init__ works but self. does not give me the values in my methods

Fix: make sure you stored the value on self in __init__. For example: self.timeout = timeout. If you wrote timeout = timeout (without self.), the value only exists inside __init__ and disappears after it returns.

Stretch Goals

If you finish the main exercise early, try one or more of these:

Stretch 1 Add a load_history() method to your ResultSaver class that reads all saved records and prints a summary — for example, the last 5 results with timestamps. Call it from main.py with a --history flag using sys.argv.

Stretch 2 Call two different endpoints in the same session. For example: if you are using RestCountries, fetch both the country info AND a random fact about that country from a second API. Wire both calls through your client class.

Stretch 3 Add a retry mechanism. If the API call fails (network error or 5xx status), wait 2 seconds and try again — up to 3 attempts. After 3 failures, give up and print a message. Use a for loop and time.sleep(2).

Glossary — New Terms This Session

Term	Plain-English Definition
API	A set of rules that lets two programs communicate over the internet. Your code sends a request; the server sends back data.
HTTP GET request	A read-only request that asks a server to send you data. requests.get(url) sends one.
JSON response	The data a server sends back, formatted as text that Python converts into a dictionary with response.json().
Status code	A number in every API response. 200 = success. 404 = not found. 500 = server error.
raise_for_status()	A requests method that automatically raises an exception if the status code signals an error (4xx or 5xx).
Environment variable	A named value stored outside your code. Read with os.environ.get("KEY"). Never put API keys in Python files.
.env file	A plain text file that holds KEY=VALUE pairs for secrets. Never committed to GitHub. Loaded by python-dotenv.
load_dotenv()	A function from python-dotenv that reads .env and puts every key into os.environ so your code can read them.
API client class	A class that wraps all communication with one API. __init__ stores config. Methods call endpoints. The caller never knows the HTTP details.
_load_existing()	A method with a leading underscore — by convention, meant for internal use only. Callers do not call it directly.